

AWS CLOUD INFRASTRUCTURE PROJECT

Distributing Traffic Across EC2 Instances using an Application Load Balancer

Configuring a Target Group and an Internet-facing ALB to balance HTTP traffic across two EC2 instances

Parameter	Value
Cloud Provider	Amazon Web Services (AWS)
Load Balancer Type	Application Load Balancer (ALB) — Layer 7
Backend Instances	2 x Windows EC2 instances (IIS web servers)
Core Services Used	EC2, Target Groups, Application Load Balancer, Security Groups
Region	US East (N. Virginia) — us-east-1

1. Objective

The objective of this project is to distribute incoming HTTP traffic across multiple EC2 instances using an AWS Application Load Balancer (ALB), so that no single server bears the entire load and the application remains available even if one instance becomes unhealthy. Two EC2 instances running web servers are registered behind a Target Group, and an internet-facing Application Load Balancer is configured to route client requests to whichever registered instance is healthy — verified here by refreshing the load balancer's DNS name and observing responses returned from both backend servers in turn.

By the end of this project, the following outcomes are achieved:

- A Target Group that groups the backend EC2 instances and continuously health-checks them.
- Two EC2 instances registered as targets, each running a web server on port 80.
- An internet-facing Application Load Balancer spanning two Availability Zones, listening on HTTP:80.
- A working DNS name for the load balancer that transparently forwards traffic to healthy targets.
- Verification that both backend instances actually serve traffic, confirming load balancing is working.

2. Theory

2.1 Elastic Load Balancing (ELB)

Elastic Load Balancing automatically distributes incoming application traffic across multiple targets, such as EC2 instances, in one or more Availability Zones. It increases the availability and fault tolerance of an application: if a registered instance fails or becomes unhealthy, the load balancer automatically stops sending it traffic and routes requests only to the remaining healthy instances.

2.2 Application Load Balancer (ALB)

An Application Load Balancer operates at Layer 7 (the application layer) of the OSI model and is best suited for HTTP/HTTPS traffic. Because it understands HTTP, it can make routing decisions based on request content — such as the URL path, hostname, or headers — and supports advanced routing to multiple target groups. In this project, the ALB is configured with a single listener that forwards all traffic to one target group.

2.3 Target Groups

A Target Group is used to route requests to one or more registered targets (EC2 instances, IP addresses, or Lambda functions) using a specified protocol and port. The load balancer's listener forwards requests to the target group, which then distributes them among its healthy, registered targets. Each target group also defines a health check — a periodic HTTP request AWS sends to each target to determine whether it should keep receiving traffic.

2.4 Health Checks

Health checks let the target group continuously monitor the state of its registered instances. A target is only sent live traffic once it reaches the Healthy threshold number of consecutive successful checks; if a target fails checks past the Unhealthy threshold, it is automatically taken out of rotation. This is the mechanism that makes the

whole architecture self-healing from a traffic-routing perspective — an unhealthy instance simply stops receiving requests until it recovers.

2.5 Listeners and Routing

A listener is a process that checks for connection requests on a configured port and protocol (e.g. HTTP on port 80). Each listener has a default action — most commonly Forward to target group — which determines what the load balancer does with requests that match no more specific rule. Listener rules can also route based on weighted target groups, letting traffic be split across multiple target groups by percentage.

2.6 Security Groups for Load Balancing

The security groups attached to the load balancer must allow inbound traffic on the listener port (HTTP/80) from clients, and the security groups on the backend EC2 instances must allow inbound traffic from the load balancer on the target port. In this project, the existing instance security groups already permit this traffic, so they were reused for the load balancer itself.

3. Architecture Overview

The architecture places an internet-facing Application Load Balancer in front of two EC2 instances running in the default VPC, spread across Availability Zones us-east-1a and us-east-1b. The ALB listens on HTTP:80 and forwards all requests to a single target group, which health-checks and load-balances traffic across both registered instances. A client only ever talks to the load balancer's DNS name — never directly to an instance's IP — so instances can be added, removed, or replaced without changing how the application is accessed.

Parameter	Value
Target Group	newtarget — Protocol: HTTP, Target type: Instance
Health Check	HTTP, path "/", 5s timeout, 30s interval, healthy threshold 5, unhealthy threshold 2
Registered Targets	windowsins (i-03f007a30741f5f46), windowsins1 (i-05bc2eab7877006bc) — port 80
Load Balancer Name	tushu — Internet-facing, IPv4
VPC / Subnets	Default VPC — us-east-1a and us-east-1b
Security Groups	launch-wizard-9, launch-wizard-10, default
Listener	HTTP:80 → Forward to target group "newtarget" (weight 1, 100%)
DNS Name	tushu-1612674092.us-east-1.elb.amazonaws.com

4. Implementation Steps

Step 1: Create a Target Group

- ▶ Navigate to EC2 → Target Groups → Create target group.
- ▶ Choose target type: Instances.
- ▶ Enter Target group name: newtarget
- ▶ Set Protocol: HTTP, Port: 80 (default health check path "/", interval 30s, timeout 5s, healthy threshold 5, unhealthy threshold 2).

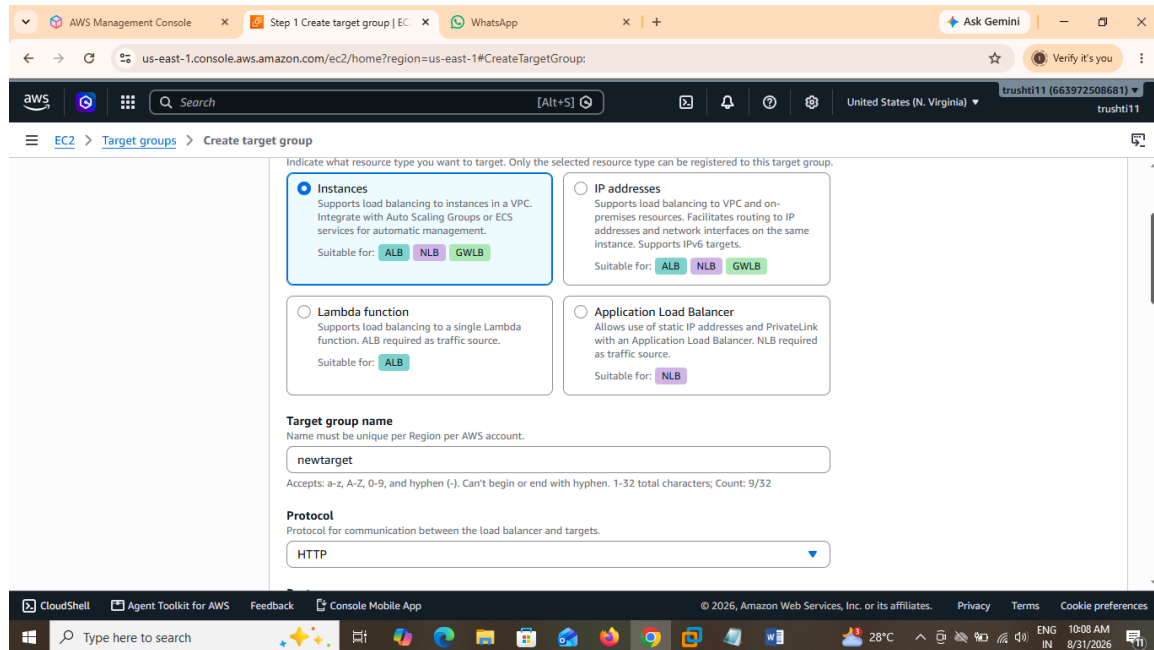


Figure 1 — Creating the target group 'newtarget' for instance targets over HTTP

Step 2: Start the Backend Instances and Register Them

- ▶ Start the two EC2 instances that will serve the website, and wait until their State shows Running.
- ▶ On the Register targets step, select both available instances (windowsins and windowsins1).
- ▶ Set the port for the selected instances to 80, then add them to the pending targets list.

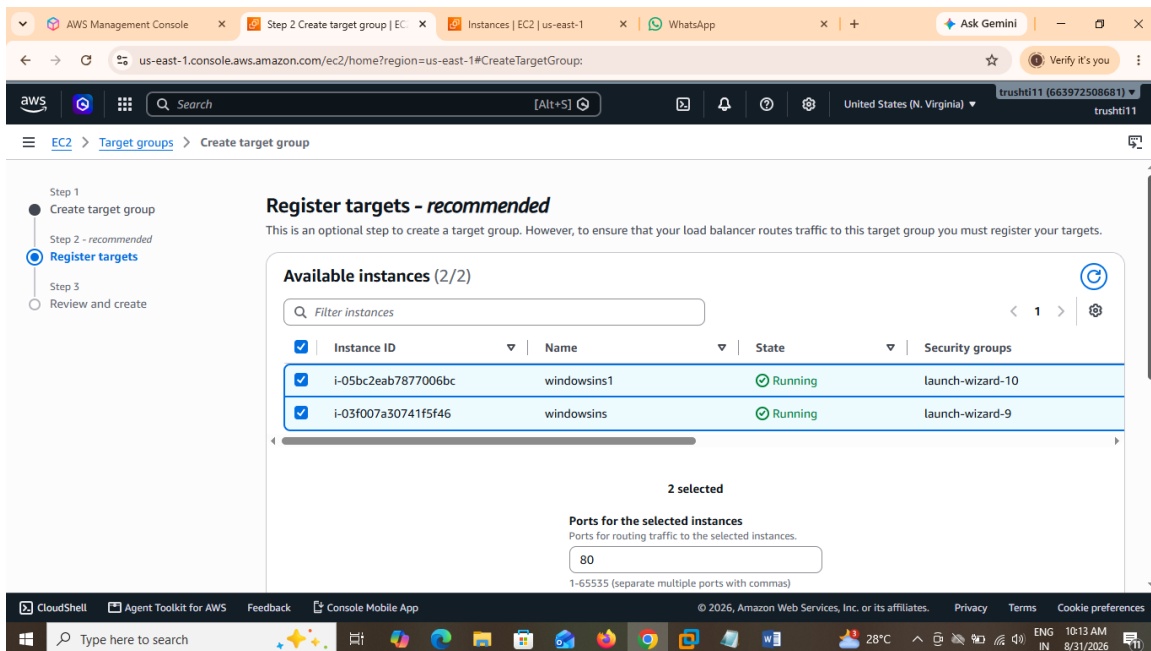


Figure 2 — Selecting both running instances and setting port 80 for target registration

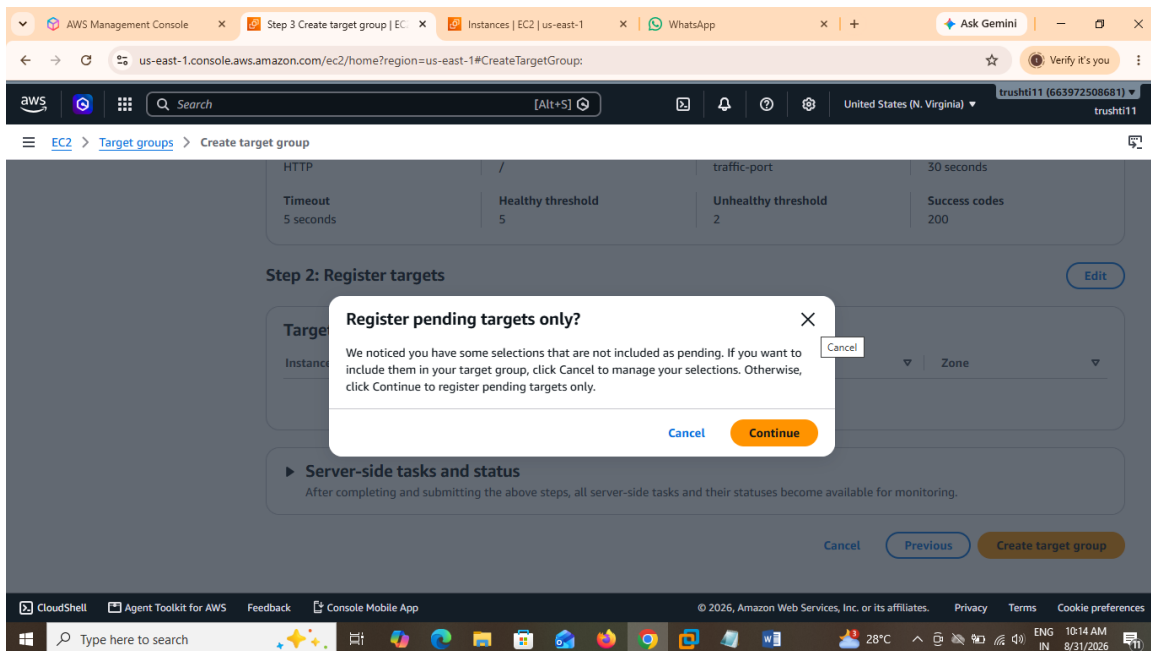


Figure 3 — Confirming pending target registration before creating the target group

Why two instances?: Registering two instances behind one target group is what allows the load balancer to spread traffic between them and keep the application available if either instance goes down.

Step 3: Create the Application Load Balancer

- ▶ Go to EC2 → Load Balancers → Create load balancer → Application Load Balancer.
- ▶ Enter a unique Load balancer name (e.g. tushu).
- ▶ Choose Scheme: Internet-facing, and IP address type: IPv4.

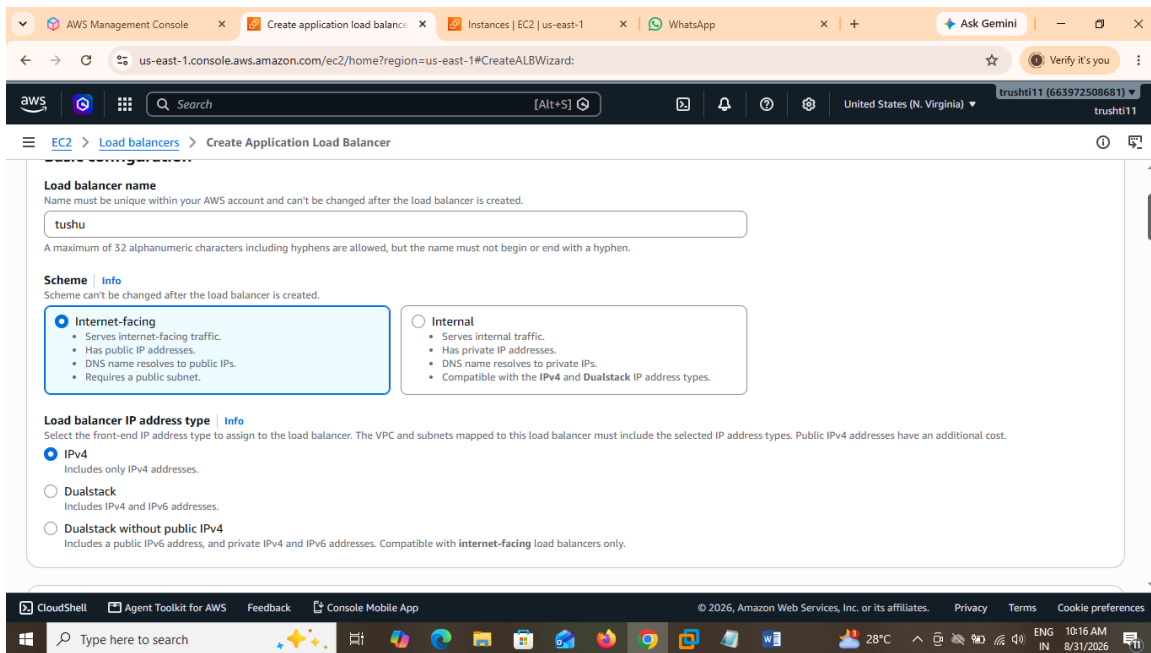


Figure 4 — Naming the load balancer 'tushu' as an internet-facing, IPv4 ALB

- Under Listeners and routing, keep the default HTTP:80 listener and set its default action to Forward to target groups, selecting newtarget as the target group.

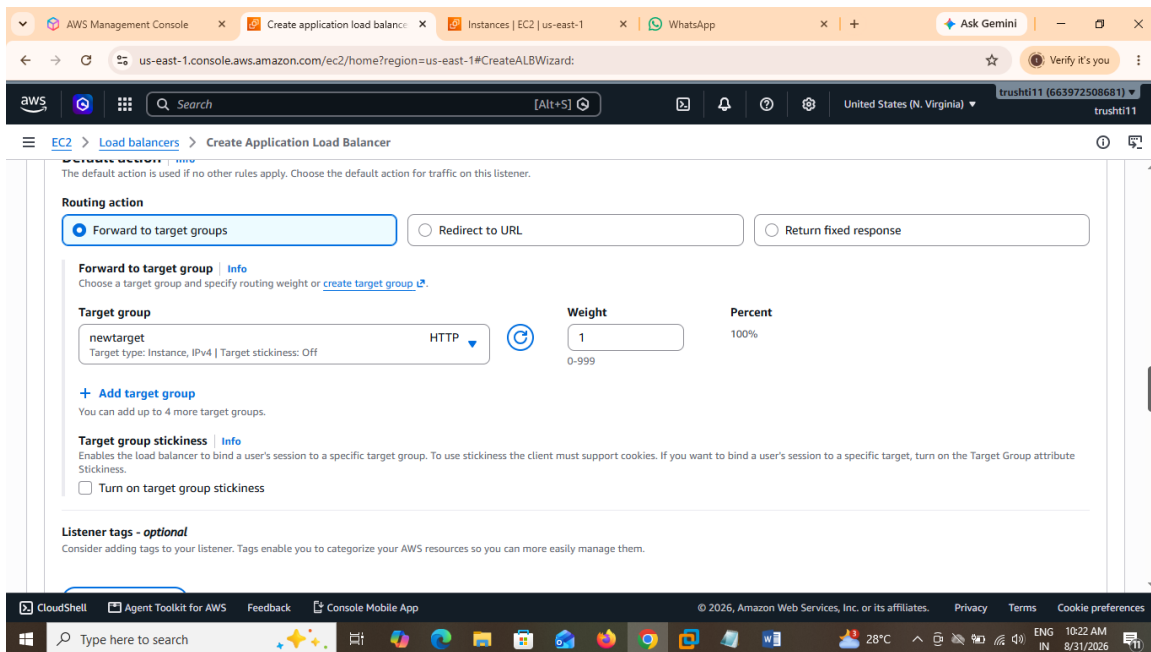


Figure 5 — Configuring the HTTP:80 listener to forward to target group 'newtarget'

- Select the VPC that hosts the instances, and choose at least two Availability Zones (us-east-1a and us-east-1b) with their subnets, so the load balancer nodes are spread across zones.

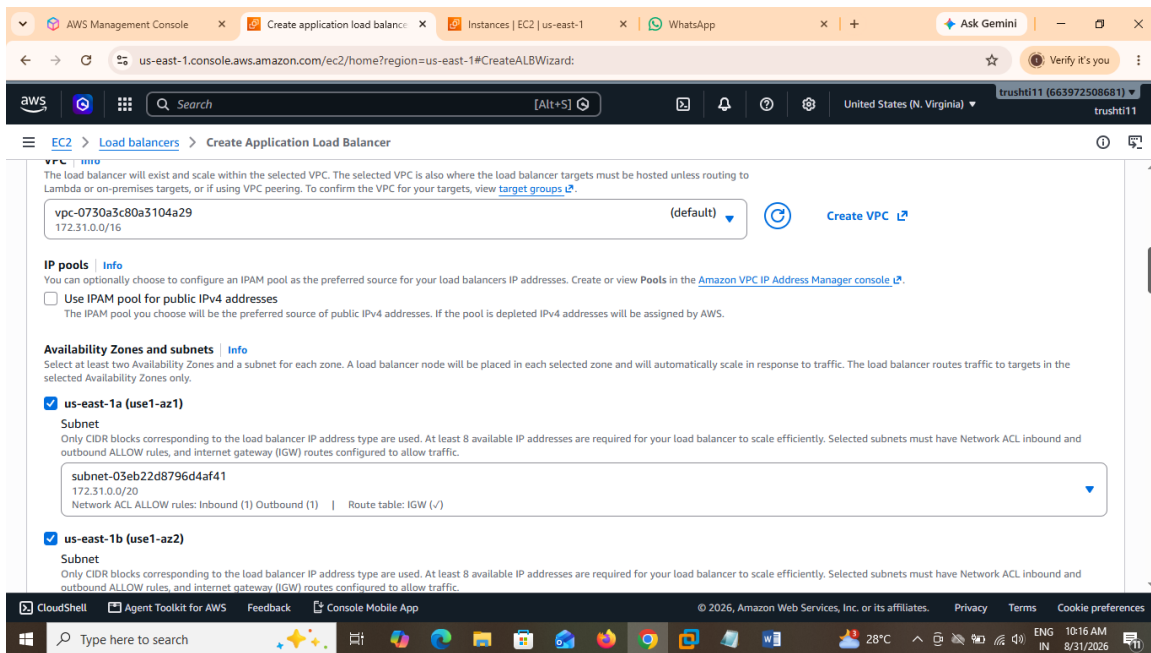


Figure 6 — Selecting the VPC and both Availability Zones/subnets for the load balancer

- ▶ Attach the security groups already used by the backend instances (launch-wizard-9, launch-wizard-10) plus the default security group, so traffic on port 80 is permitted end-to-end.

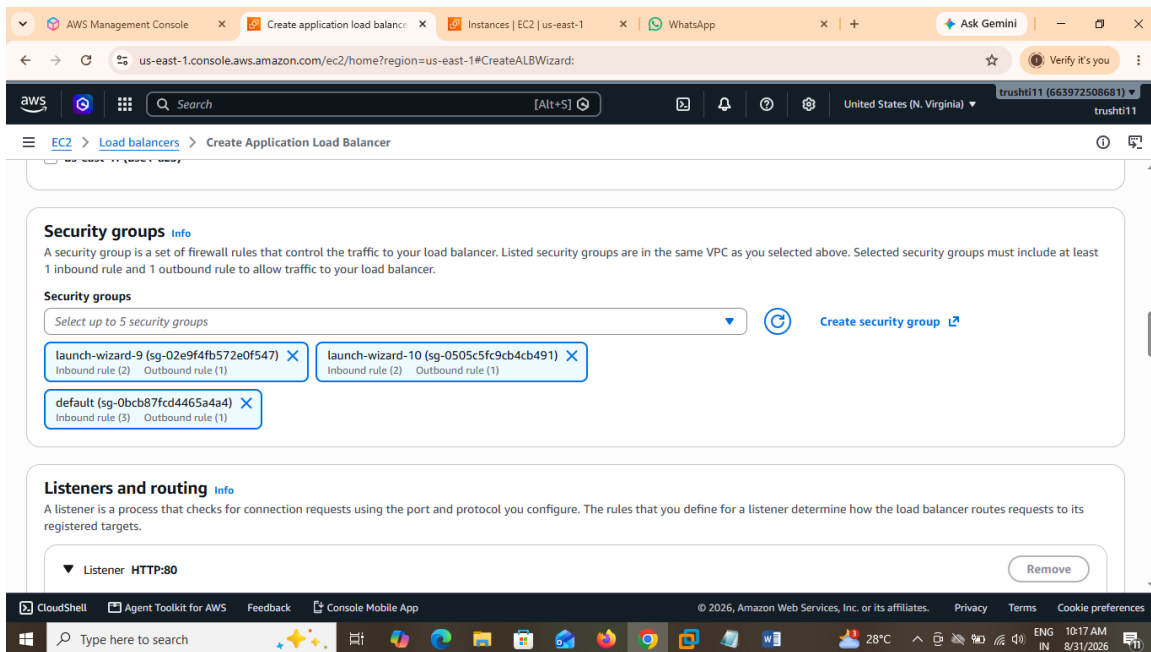


Figure 7 — Attaching security groups and reviewing the HTTP:80 listener before creation

- ▶ Review the configuration and click Create load balancer.

Step 4: Register Targets from the Target Group

- ▶ Open the newtarget target group → Register targets.
- ▶ Select both available instances again, confirm the port (80), and click Register pending targets.

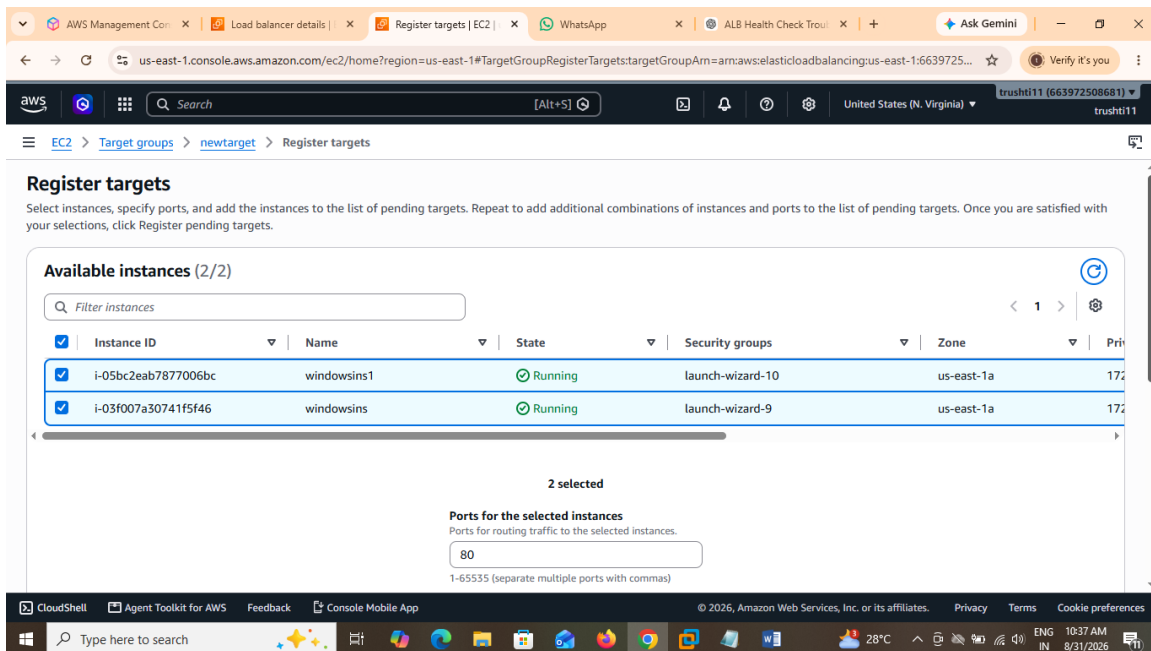


Figure 8 — Registering both instances as targets from the Target Group console

Step 5: Verify the Load Balancer and Copy Its DNS Name

- ▶ Open EC2 → Load Balancers and confirm the load balancer's State shows Active.
- ▶ Wait for the Target group's target health status to show healthy / in-service for both instances.
- ▶ Copy the load balancer's DNS name (A record) to use in the browser.

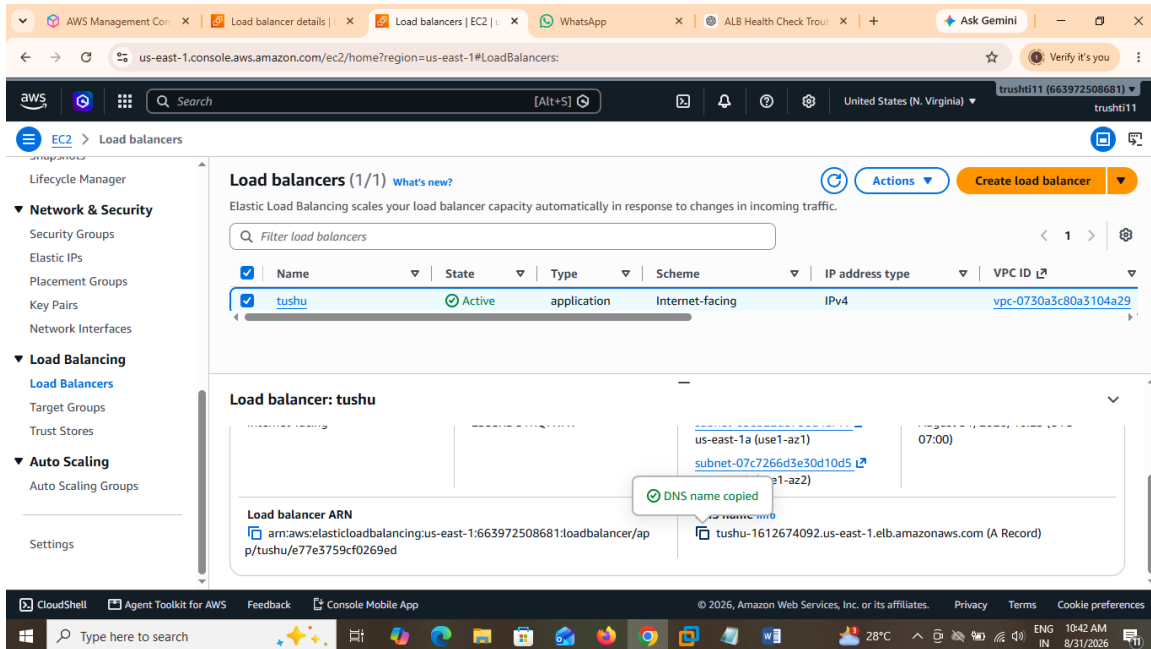


Figure 9 — Load balancer 'tushu' is Active; DNS name copied for testing

Step 6: Test Load Balancing in the Browser

- ▶ Paste the DNS name into a browser address bar and load the page.

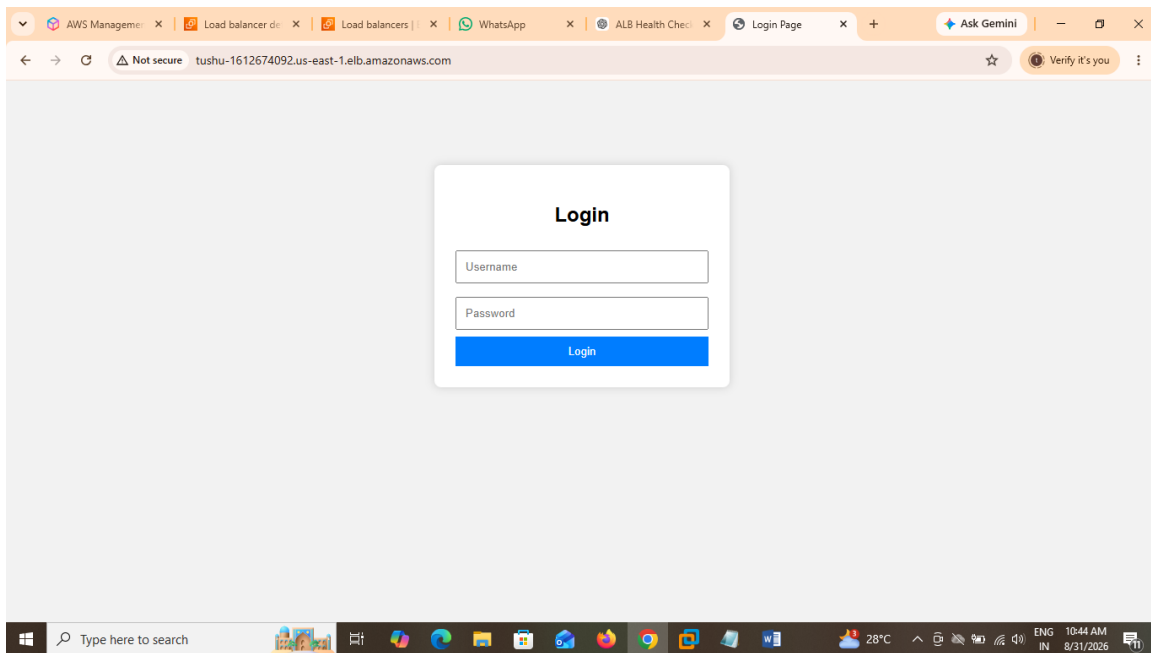


Figure 10 — The website loaded via the ALB's DNS name, served by the first backend instance

- Refresh the same DNS name again — the Application Load Balancer routes the new request to the other healthy target in the target group.

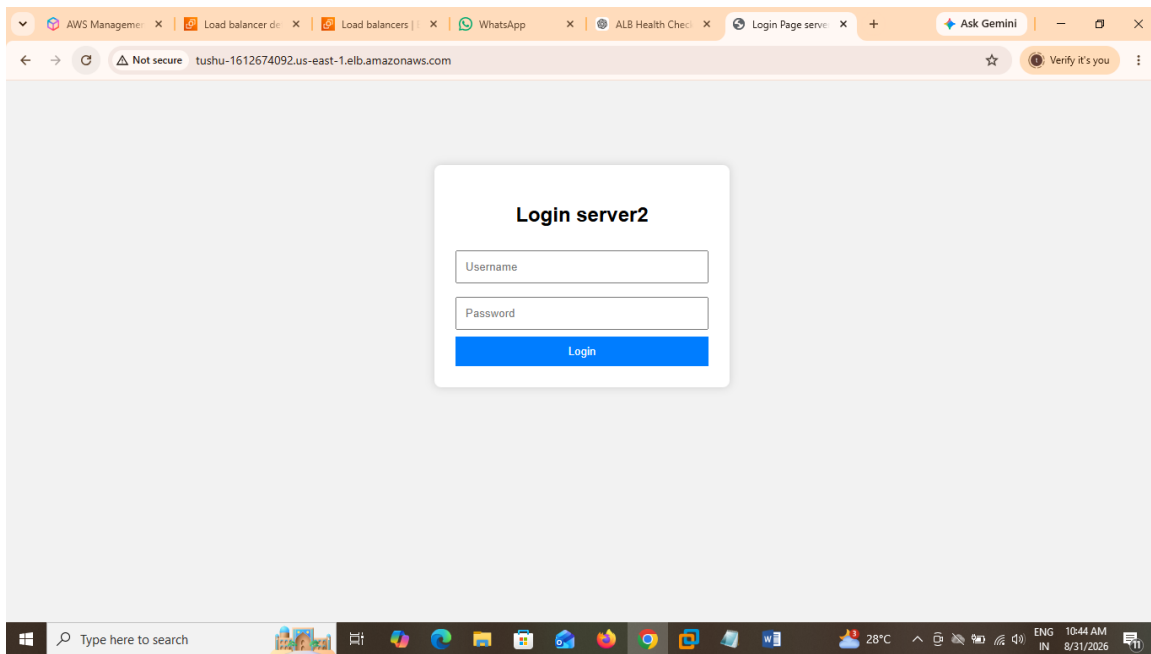


Figure 11 — The same DNS name now returning a response from the second backend instance ('server2')

The subtitle changes from the first page load to the next ("Login" vs. "Login server2") while the URL in the address bar stays exactly the same. This confirms that the Application Load Balancer is actively distributing requests between both registered EC2 instances rather than always hitting the same server.

5. Conclusion

This project successfully demonstrates how to distribute HTTP traffic across multiple EC2 instances using an AWS Application Load Balancer. A target group was created to track and health-check two backend instances, and an internet-facing ALB was configured to forward all incoming HTTP:80 traffic to that target group. Testing against the load balancer's DNS name confirmed that requests were served by both registered instances in turn, proving that traffic is genuinely being balanced rather than sent to a single fixed server.

The key benefit of this approach is availability and scalability: clients only ever need the load balancer's single DNS name, while the set of backend instances behind it can grow, shrink, fail, or be replaced without any change to how the application is accessed. This is the same pattern used to front production web applications and pairs naturally with an Auto Scaling Group, which can register and deregister instances from the target group automatically as capacity changes.

6. Key Learnings

- How a Target Group registers EC2 instances and continuously health-checks them before sending them traffic.
- The difference between registering targets during target-group creation versus registering them afterward from the Target Groups console.
- How to configure an internet-facing Application Load Balancer, including scheme, IP address type, Availability Zones, and security groups.
- How a listener's default action forwards traffic to a target group, and how routing weight controls the split across multiple target groups.
- Why the load balancer's DNS name — rather than an instance's IP — should always be used to access the application.
- How to verify that load balancing is actually occurring, by observing responses from different backend instances across repeated requests.